

Assignment 4

Colton Brewer

Link to the github repository:

[Colt-the-analyst/Assignment_4_471: Repository for the working of assignment 4 for VUW 2026 semester 2 DATA471](#)

Exercise 1

1.)

Hyndman and Fan introduced 9 different definitions of sample quantiles because the quantile can fall between two observations. Each of these use a different rule for either selecting or interpolation between observations.

1. Let N be the count of observations in the ordered sample, and q be the desired quantile. q is rounded up to the nearest x/N for integer x . the quantile takes the x th value in the ordered sample.
2. Same as (1.), but in the boundary case where there exists integer x for $q = x/N$, the quantile takes the mean of the x th and $(x+1)$ th value in the ordered sample.
3. Same as (1.), but instead of rounding up to the nearest x/N for integer x , it rounds to the closest x/N (which may be less than q). For situations where $q - x/N = (x+1)/n - q$, the quantile takes the x th value in the ordered sample when x is even, otherwise it takes the $(x+1)$ th value.
4. Let N be the count of observations in the ordered sample, and q be the desired quantile. if $q = x/N$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists integer x such that $x/N < q < (x+1)/N$. let s_x and $s_{(x+1)}$ represent the x th and $(x+1)$ th value in the ordered sample. The quantile takes the value $(qN-x)(s_{(x+1)}-s_x)+s_x$.
5. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $(x-0.5)/N$. If $q = (x-0.5)/N$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer x such that $(x-0.5)/N < q < (x+0.5)/N$. Let s_x and $s_{(x+1)}$ represent the x th and $(x+1)$ th values in the ordered sample. The quantile takes the value $(qN-x+0.5)(s_{(x+1)}-s_x)+s_x$.

6. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $x/(N+1)$. If $q = x/(N+1)$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer such that $x/(N+1) < q < (x+1)/(N+1)$. The quantile takes the value $(q(N+1)-x)(s_{x+1}-s_x)+s_x$.
7. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $(x-1)/(N-1)$. If $q = (x-1)/(N-1)$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer x such that $(x-1)/(N-1) < q < x/(N-1)$. The quantile takes the value $(q(N-1)-x+1)(s_{x+1}-s_x)+s_x$.
8. Let N be the count of observations in the ordered sample, and q be the desired quantile. the x th value in the ordered sample is assigned the position $(x-1/3)/(N+1/3)$. If q equals this position for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer x such that $(x-1/3)/(N+1/3) < q < (x+2/3)/(N+1/3)$. The interpolation formula for this method is $(q(N+1/3)-x+1/3)(s_{x+1}-s_x)+s_x$.
9. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $(x-3/8)/(N+1/4)$. If q equals this position for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer such that $(x-3/8)/(N+1/4) < q < (x+5/8)/(N+1/4)$. The quantile takes the value $(q(N+1/4)-x+3/8)(s_{x+1}-s_x)+s_x$.

The first three result in quantiles that always fall within the sample space, but the remaining methods involve interpolation, resulting in the only restriction of the quantiles is that they fall between the highest and lowest values in the sample. For 5, 6, 8 and 9, values of q close to 0 or 1 can fall outside the positions assigned to the observations. In these cases, the quantile takes the value of the sample minimum or maximum. Note that the differences between these methods grow smaller as sample size increases. While R uses type 7, Hyndman and Fan recommend type 8 as it is an approximately median unbiased estimator that does not depend on a particular underlying distribution. Because of this, setting the type arguments to 8 instead of the default 7 is used when calculating the quantiles.

2.)

A biased estimator is a formula where the expected value does not equal the true population parameter. The exponential quantile function is $Q(p) = -\log(1-p)/\lambda$, meaning that the population quartiles are $Q(0.25) = -\log(0.75)/\lambda$ and $Q(0.75) = -\log(0.25)/\lambda$. The interquartile range is $Q(0.75) - Q(0.25) = \log(3)/\lambda$. Solving for λ , $\lambda_{IQR} = \log(3)/(Q(0.75) - Q(0.25))$.

The estimated bootstrap bias is calculated as:

$$\widehat{\text{Bias}} = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}^{*(b)} - \hat{\lambda}_{IQR}$$

Where B is the number of bootstrap replications. Substituting this bias formula into the adjusted estimate for lambda, taking bias into account gives:

$$\begin{aligned} \tilde{\lambda} &= \hat{\lambda}_{IQR} - \widehat{\text{Bias}} \\ &= 2\hat{\lambda}_{IQR} - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}^{*(b)} \end{aligned}$$

Where $\tilde{\lambda}$ is the estimate that takes bootstrap bias into account.

```
# Create function for lambda using the interquartile range (type 8) method
lambda_iqr <- function(x) {
  sample_iqr <- IQR(
    x,
    type = 8
  )

  return(log(3)/sample_iqr)
}

# Create function for bootstrap sample
lambda_iqr_corrected <- function(x, size = 200) {
  lambda_iqr_initial <- lambda_iqr(x)
  lambda_bootstrap <- replicate(size, {
    bootstrap_sample <- rexp(
      n = length(x),
      rate = lambda_iqr_initial
    )
    lambda_iqr(bootstrap_sample)
  })

  # Estimate bias
  estimated_bias <- mean(lambda_bootstrap) - lambda_iqr_initial

  # Create the first order bias corrected estimate
  lambda_corrected <- lambda_iqr_initial - estimated_bias
```

```
    return(lambda_corrected)
  }
```

3.)

A parametric bootstrap was used where each sample was generated from an exponential distribution using the λ which was found through the interquartile range method. This method was chosen because a fundamental assumption of the exercise was that the sample followed an exponential distribution.

```
# Create the ml estimator function
lambda_ml <- function(x) {
  1/mean(x)
}

# Run the Monte Carlo experiment
run_simulation <- function(true_lambda, n, M = 1000, size = 200) {
  estimates <- replicate(M, {

    x <- rexp(
      n = n,
      rate = true_lambda
    )

    c(
      ML = lambda_ml(x),
      IQR = lambda_iqr(x),
      Bootstrap = lambda_iqr_corrected(x, size = size)
    )
  })

  estimates <- as.data.frame(t(estimates))

  mse_ml <- mean(
    (estimates$ML - true_lambda)^2
  )

  mse_iqr <- mean(
    (estimates$IQR - true_lambda)^2
  )
}
```

```

mse_bootstrap <- mean(
  (estimates$Bootstrap - true_lambda)^2
)

tibble(
  lambda = true_lambda,
  n = n,
  estimator = c("ML", "IQR", "Bootstrap"),
  MSE = c(
    mse_ml, mse_iqr, mse_bootstrap
  )
)
}

# Run every combination
parameter_space <- expand_grid(
  lambda = c(0.1, 1, 10),
  n = c(5, 30, 100)
)

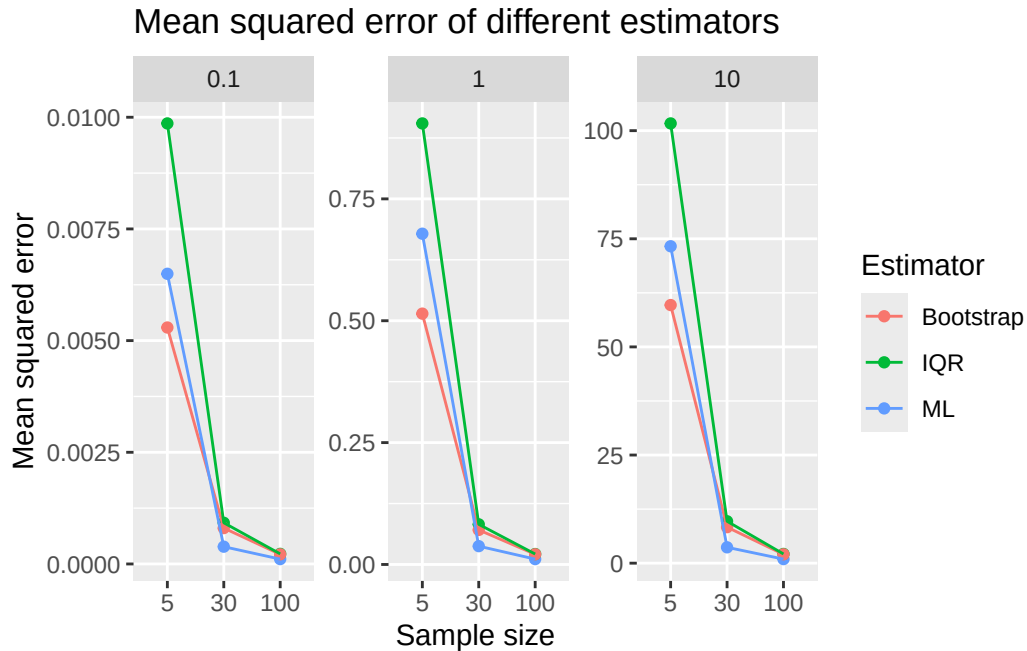
monte_carlo_results <- parameter_space |>
  pmap_dfr(
    \(lambda, n) {
      run_simulation(
        true_lambda = lambda,
        n = n,
        M = 1000,
        size = 200
      )
    }
  )

# Show the data in a wide format
(monte_carlo_results_wide <- monte_carlo_results |>
  pivot_wider(
    names_from = estimator,
    values_from = MSE
  )) |>
  pander()

```

lambda	n	ML	IQR	Bootstrap
0.1	5	0.006496	0.009862	0.005293
0.1	30	0.0003868	0.0009219	0.0008032
0.1	100	0.0001085	0.0002258	0.0002156
1	5	0.6785	0.9048	0.5146
1	30	0.03756	0.08235	0.07085
1	100	0.01098	0.02141	0.02083
10	5	73.27	101.7	59.68
10	30	3.667	9.691	8.321
10	100	0.9371	2.128	2.041

```
# Plot the results
monte_carlo_results |>
  ggplot(
    aes(
      x = factor(n),
      y = MSE,
      colour = estimator,
      group = estimator
    )
  ) +
  geom_point() +
  geom_line() +
  facet_wrap(
    ~ lambda,
    scales = "free_y"
  ) +
  labs(
    title = "Mean squared error of different estimators",
    x = "Sample size",
    y = "Mean squared error",
    colour = "Estimator"
  )
```



The 1,000 Monte Carlo replications for each combination of n and λ were chosen because it is enough to produce reasonably stable MSE values while keeping the computation doable. Note that each replication also has 200 bootstrap samples. The MSE decreases as the sample size increases, meaning that all estimates improve as more data becomes available. The maximum likelihood estimator tends to perform the best, having the lowest MSE. The improvement that the bias correction has on the method of interquartile range has decreasing returns as sample size increases.

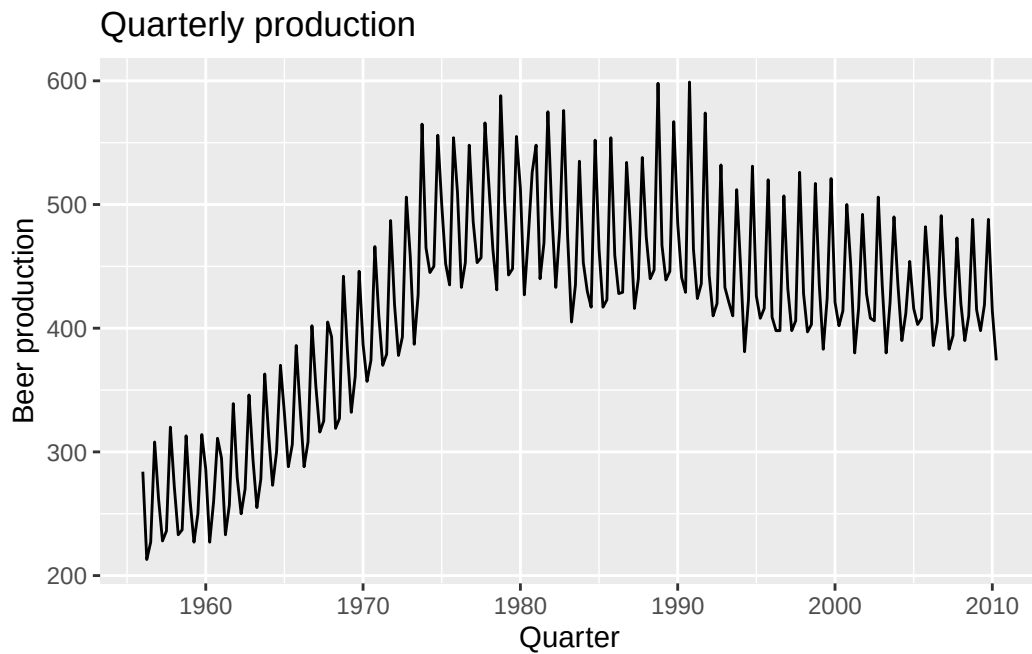
Exercise 2

1.)

The frequency of the ausbeer time series is 4 observations in each year, meaning that it's quarterly data. It runs from Q1 1956 to Q2 2010, which is confirmed in the package documentation. The frequency can also be seen in the time series plot below, as there is a clear seasonal pattern occurring every 4 instances.

```
# Time series line graph
autoplot(ausbeer) +
  labs(
    title = "Quarterly production",
    x = "Quarter",
```

```
y = "Beer production"  
)
```



2.)

The series can be seen having a strong linear increasing trend with seasonal pattern until 1974. Beyond that, an ever so slightly decreasing trend follows, with more annual variations than before 1974. The long term trend is present, but it isn't consistent through this historical period, being an increase before a smaller decline starting around 1975.

3.)

Seasonality is present through this time series, where the largest trough in a period is in quarter 2, before increasing the next two subsequent quarters to reach the largest peak in quarter 4, before decreasing in quarter 1. This pattern is followed in most years, but there are instances where Q3 has the largest trough.

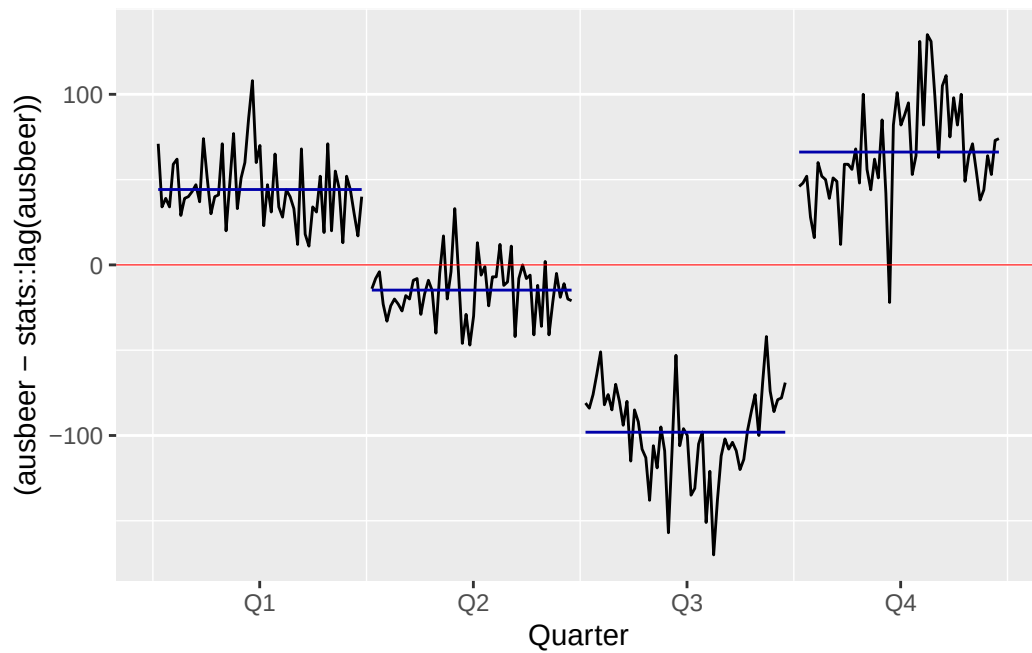
4.)

The time series displays heteroscedasticity meaning that the variability isn't constant over time. In the earlier years when there was a relatively constant increase (accounting for seasonality) the variability remains steadfast. Once the series transitions away from the steady increase over time, the variability also increases. Near the end of this series, it seems to slowly decrease. A log transformation may help stabilize this irregular variance to some degree, but this method has best results when variability scales with the observed outcome variable (beer production), which isn't the case here since variation was consistent in the earlier increasing years.

5.)

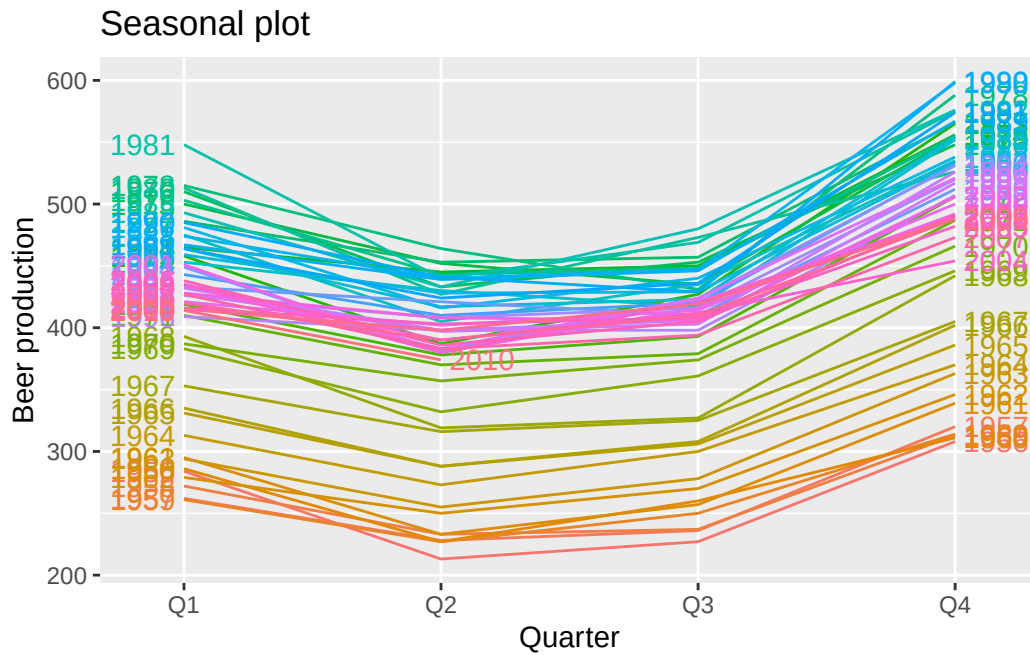
One feature that's of note is that there is one and only one quarter 4 difference going into quarter 1, being the 1980 Q4 of beer production going into quarter 1 in 1981. While the quarter 3 changes going into quarter 1 go through the most changes (visually looking more like a random walk than white noise), the quarter 3 change into quarter 4 in 1980 is very high relative to neighboring years.

```
(ausbeer - stats::lag(ausbeer)) |>
  ggsubseriesplot() +
  geom_hline(
    yintercept = 0,
    colour = "red",
    linewidth = 0.2
  )
```



6.)

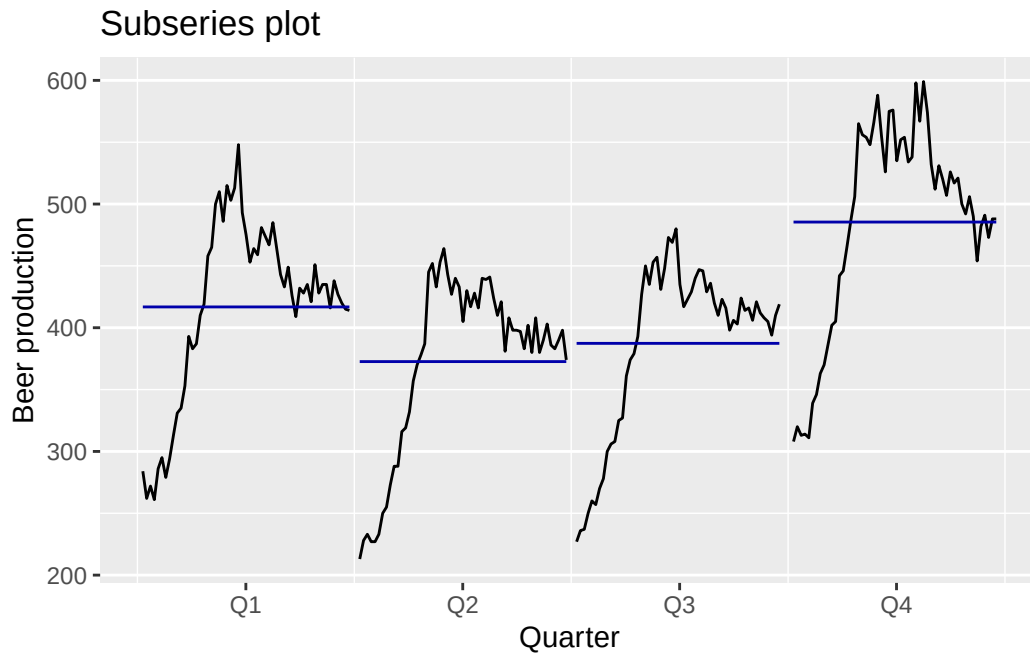
```
ausbeer |> ggseasonplot(
  year.labels = TRUE,
  year.labels.left = TRUE
) +
  labs(
    title = "Seasonal plot",
    x = "Quarter",
    y = "Beer production"
  )
```



There's a strong pattern of Q4 being the highest and Q1 being the second highest in terms of beer production. Q2 and Q3 are relatively interchangeable.

7.)

```
ausbeer |> ggsubseriesplot() +
  labs(
    title = "Subseries plot",
    x = "Quarter",
    y = "Beer production"
  )
```



This plot shows that the ranking of quarters from highest beer production to lowest is 4, 1, 3, then 2. Seasonal pattern isn't stable over time, as there is a higher variance shown after the initial, stable increase. If it were stable over this period, there would be much less variance.

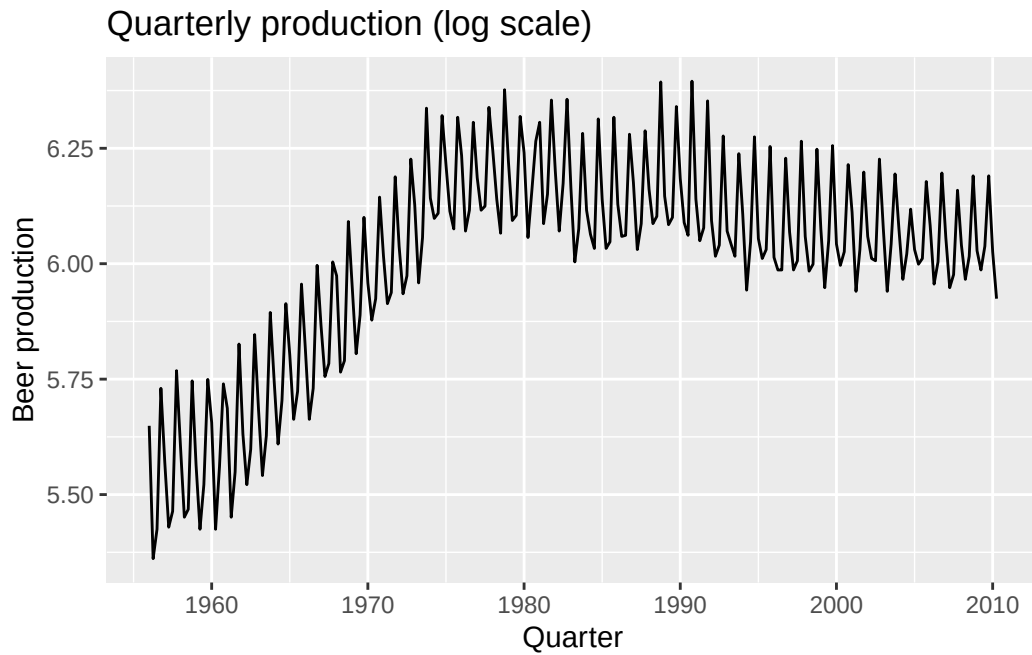
8.)

Detrending the entire time series would be inappropriate. This is because the earlier years would drive the coefficient of time up, making the detrended model slanted slightly up in the beginning and slightly down in the end. An appropriate alternative would be to research why there was a change in trend in the mid seventies, talk to stakeholders as to the likelihood of either a similar event occurring in the future where a downward shift in trend could be expected, or if the trend pre 1974 could return. In either case, the training data could be weighted more heavily either pre 1974 or post 1974 depending on their input.

While including a log transformation to account for the heteroscedasticity may sound good on paper, the issue is that it doesn't fit the constant variance shown in the beginning of the time series. More time series focused transformations such as taking the differences of subsequent values won't be necessary, as these can be handled implicitly by many models.

```
# Time series line graph
autoplot(log(ausbeer)) +
  labs(
    title = "Quarterly production (log scale)",
```

```
x = "Quarter",  
y = "Beer production"  
)
```



Note that there are time series models such as TBATS that can automatically handle the requirements of both removing trends and apply transformations, while more standard models such as exponential smoothing and ARIMA can account for trends by including a trend component in the model and adding a constant to the first differences in the model respectively.